Institut Teknologi Bandung

Directorate of Continuing Professional Education

in partnership with

CHAPTER 15

Language Models and the Transformer

From predicting one character of Shakespeare to fine-tuning a 124-million-parameter pretrained encoder — and the one mechanism that made the difference.

Duration 4 hours (3 sessions) **Instructor** Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 15

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Define a **language model** as p(token | past tokens), and explain why predicting one token at a time makes text generation tractable at all.
2. Build and **sample from** an autoregressive character-level model, including the inference loop that has no counterpart in training.
3. Build **sequence-to-sequence** translation with RNNs, and state the two limits that killed the approach.
4. Derive **attention** from the problem it solves — score, softmax, weighted sum — and read query/key/value fluently.
5. Assemble the **Transformer encoder and decoder** blocks, and explain every part: residuals, layer normalization, causal masking, feedforward.
6. Explain why the Transformer **needs positional embeddings**, and what happens numerically when you leave them out.
7. **Fine-tune a pretrained RoBERTa** on a classification task with KerasHub.
8. Explain what attention is doing **geometrically**, and why interpolation produces both generalization and hallucination.

BOOK
Chapter 15 — full text

NOTEBOOK
01_shakespeare_language_model.ipynb

NOTEBOOK
02_seq2seq_rnn_translation.ipynb

NOTEBOOK
03_attention_from_scratch.ipynb

NOTEBOOK
04_transformer_translation.ipynb

NOTEBOOK
05_finetuning_roberta.ipynb

REPO
Author's official notebook

01

The language model

One idea, small enough to fit on a line: p(token | past tokens).

SECTION 15.1

Classification was the easy case

The previous chapter classified movie reviews. Text went **in**; a single floating-point number came **out**.

That output shape is unusually forgiving. Binary classification needs one number. *N*-way classification needs *N*. Both are small, fixed, and known in advance.

Question answering and translation are not like that. They need a model that **generates text as output**. Just as we needed tokenizers and embeddings to handle text on the way **in**, we need new machinery to produce it on the way **out**.

> We do not start from scratch. An integer sequence is still the natural numeric representation for text — we **detokenize** by running the mapping in reverse.

SECTION 15.1

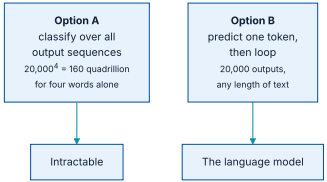
The obvious approach does not survive arithmetic

20,000
vocabulary size

160 quadrillion
possible 4-word sequences

> atoms in the universe
possible 20-word sequences

Twenty thousand to the fourth power is 1.6×10^{17}. No model design rescues this — the output layer alone would overwhelm any compute budget that will ever exist.



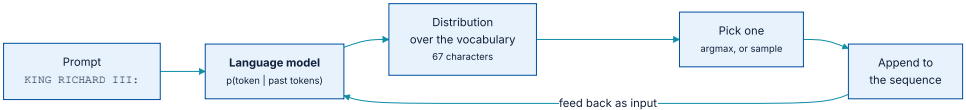
Two ways to produce a sequence, and only one of them is finite.

A language model predicts one token at a time

A **language model** learns a straightforward but deep probability distribution: **p(token | past tokens)**

Given every token observed up to a point, it outputs a probability distribution over all tokens that could come next. With a 20,000-word vocabulary, the model needs to produce **20,000 numbers** — not 160 quadrillion.

And yet, by predicting the next token repeatedly, feeding each prediction back in as input, we have built a model that can generate text of **any length at all**.



The autoregressive loop: the output at one step is the input at the next.

A concrete case: Shakespeare, one character at a time

listing 15.1 PYTHON

```python
import keras

filename = keras.utils.get_file(
    origin=(
        "https://storage.googleapis.com/download.tensorflow.org/"
        "data/shakespeare.txt"
    ),
)
shakespeare = open(filename, "r").read()
```

A character-level model keeps the whole example small enough to train in about two minutes, while demonstrating every mechanism a billion-parameter model uses.

SECTION 15.1.1

What the corpus looks like

OUTPUT

```
>>> shakespeare[:250]
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.
```

Note what the model will have to learn: speaker names in capitals, a colon, a newline, then verse. **None of that is given to it.** It is all structure to be discovered from next-character prediction alone.

Features and labels differ by exactly one character

listing 15.2 PYTHON

```python
import tensorflow as tf

sequence_length = 100

def split_input(input, sequence_length):
    for i in range(0, len(input), sequence_length):
        yield input[i : i + sequence_length]

features = list(split_input(shakespeare[:-1], sequence_length))
labels = list(split_input(shakespeare[1:], sequence_length))
dataset = tf.data.Dataset.from_tensor_slices((features, labels))
```

`shakespeare[:-1]` against `shakespeare[1:]` — the labels are the features **shifted one step into the future**. That single line is the entire supervision signal.

SECTION 15.1.1

The offset, seen in one sample

OUTPUT

```
>>> x, y = next(dataset.as_numpy_iterator())
>>> x[:50], y[:50]
(b"First Citizen:\nBefore we proceed any further, hear",
 b"irst Citizen:\nBefore we proceed any further, hear ")
```

At every position in the sequence, the label is the **next** character. One 100-character input yields **100 supervised predictions**, not one — which is why this trains quickly despite the tiny corpus.

A character-level vocabulary needs 67 entries

listing 15.3 PYTHON

```python
from keras import layers

tokenizer = layers.TextVectorization(
    standardize=None,
    split="character",
    output_sequence_length=sequence_length,
)
tokenizer.adapt(dataset.map(lambda text, labels: text))
```

OUTPUT

```
>>> vocabulary_size = tokenizer.vocabulary_size()
>>> vocabulary_size
67
```

Sixty-seven symbols cover the complete works. Compare the 20,000-word vocabulary of chapter 14 — this is the **vocabulary/sequence-length trade-off** in its extreme form.

The model: embed, recur, project

listing 15.4 PYTHON

```
embedding_dim = 256
hidden_dim = 1024

inputs = layers.Input(shape=(sequence_length,), dtype="int", name="token_ids")
x = layers.Embedding(vocabulary_size, embedding_dim)(inputs)
x = layers.GRU(hidden_dim, return_sequences=True)(x)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(vocabulary_size, activation="softmax")(x)
model = keras.Model(inputs, outputs)
```

`return_sequences=True` is essential: we want a prediction at **every position**, not just at the end. The final `Dense` outputs a distribution over all 67 possible next characters.

Four million parameters, and where they sit

OUTPUT

```
>>> model.summary()
Model: "functional"
+---------------------------+---------------------+-------------+
| Layer (type)              | Output Shape        |     Param # |
+---------------------------+---------------------+-------------+
| token_ids (InputLayer)    | (None, 100)         |           0 |
| embedding (Embedding)     | (None, 100, 256)    |      17,152 |
| gru (GRU)                 | (None, 100, 1024)   |   3,938,304 |
| dropout (Dropout)         | (None, 100, 1024)   |           0 |
| dense (Dense)             | (None, 100, 67)     |      68,675 |
+---------------------------+---------------------+-------------+
 Total params: 4,024,131 (15.35 MB)
```

Almost **98% of the parameters are in the GRU**. In the Transformer models later in this chapter the balance shifts dramatically — worth remembering the shape of this table.

It is still classification — 6,400 of them per batch

listing 15.5 PYTHON

```python
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["sparse_categorical_accuracy"],
)
model.fit(training_data, epochs=20)
```

The model still solves a classification problem — it just makes one classification per **token**. A batch of 64 samples with 100 characters each produces **6,400 individual labels**.

Keras averages loss and accuracy first across each sequence, then across the batch. After 20 epochs the model predicts the next character correctly about **70%** of the time.

Generation needs surgery on the trained model

A model called in a feedback loop, where its output at one step becomes its input at the next, is called **autoregressive**.

During training

Fixed sequence length of 100 tokens. The GRU's state is handled **implicitly** inside the layer call.

During generation

One token at a time, and the GRU state must be **explicitly** returned so it can be passed back in on the next call.

The computational structure is identical, so we build a second model with modified inputs and outputs and **copy the weights across**.

The inference model, with state made explicit

listing 15.6 PYTHON

```python
inputs = keras.Input(shape=(1,), dtype="int", name="token_ids")
input_state = keras.Input(shape=(hidden_dim,), name="state")

x = layers.Embedding(vocabulary_size, embedding_dim)(inputs)
x, output_state = layers.GRU(hidden_dim, return_state=True)(
    x, initial_state=input_state
)
outputs = layers.Dense(vocabulary_size, activation="softmax")(x)

generation_model = keras.Model(
    inputs=(inputs, input_state),
    outputs=(outputs, output_state),
)
generation_model.set_weights(model.get_weights())
```

`shape=(1,)` — one character in. `return_state=True` plus an `initial_state` input — the recurrent state now travels **through the function signature** rather than inside the layer.

Priming: replaying a prompt to reach the right state

listing 15.7 PYTHON

```python
tokens = tokenizer.get_vocabulary()
char_to_id = dict(zip(tokens, range(vocabulary_size)))
id_to_char = dict(zip(range(vocabulary_size), tokens))

prompt = "\nKING RICHARD III:\n"
input_ids = [char_to_id[c] for c in prompt]

state = keras.ops.zeros(shape=(1, hidden_dim))
for token_id in input_ids:
    inputs = keras.ops.expand_dims([token_id], axis=0)
    predictions, state = generation_model.predict((inputs, state), verbose=0)
```

When the last character of the prompt goes in, the returned state summarises the **entire prompt**, and the returned prediction is already the distribution for the first generated character.

The sampling loop

listing 15.8 PYTHON

```python
import numpy as np

generated_ids = []
max_length = 250
for i in range(max_length):
    next_char = int(np.argmax(predictions, axis=-1)[0])
    generated_ids.append(next_char)
    inputs = keras.ops.expand_dims([next_char], axis=0)
    predictions, state = generation_model.predict((inputs, state), verbose=0)

output = "".join([id_to_char[token_id] for token_id in generated_ids])
print(prompt + output)
```

Three lines of substance: take the most likely character, append it, feed it back. The state persists across iterations and carries everything the model knows about what it has already written.

What two minutes of training produces

OUTPUT

```
KING RICHARD III:
Stay, men! hear me speak.

FRIAR LAURENCE:
Thou wouldst have done thee here that he hath made for them?

BUCKINGHAM:
What straight shall stop his dismal threatening son,
Thou bear them both. Here comes the king;
Though I be good to put a wife to him,
```

Not the next great tragedy. But look at what a **next-character** objective produced: correctly spelled words, speaker names in capitals followed by a colon and a newline, blank lines between speeches, and roughly verse-length lines.

> We trained on the narrow problem of guessing one character. We are using it for the far broader problem of **open-ended text generation**. That gap is the point.

Replace GRU with Bidirectional(GRU) and watch it break

WHAT YOU CHANGE	TRAINING ACCURACY	GENERATION
`GRU(hidden_dim, return_sequences=True)`	≈ 70% after 20 epochs	Shakespeare-like text
`Bidirectional(GRU(...))`	**above 99% immediately**	**stops working entirely**

With a bidirectional layer, information from the *next* token reaches the prediction of the current one. The model reads the label off its own input — the problem becomes trivial, and nothing generalisable is learned.

> A training metric that suddenly looks *too good* is the most reliable signal of leakage there is. Chapter 5's lesson, in a new costume.

There is now logic that exists only at inference time

Every model so far in this book answered `model.predict()`. This one does not.

- There is an entire **loop**, and a non-trivial amount of logic, that exists **only at inference time**.
- State looping inside the GRU cell happens during both training and inference — that part is symmetric.
- But at **no point during training** do we feed the model's own predicted labels back to it as input. That happens only when generating.

This asymmetry — training on ground-truth prefixes, generating from self-produced prefixes — has a name in the literature (*exposure bias*) and is one reason generated text degrades over long outputs.

02

Sequence-to-sequence learning

Two sequences, one model: translation, and the encoder/decoder template.

The encoder/decoder template

Encoder
Turns the **whole** source sequence into an intermediate representation. It may look forward and backward freely.

Decoder
The language-model setup from section 15.1, with one addition: it also sees the encoder's representation of the source.

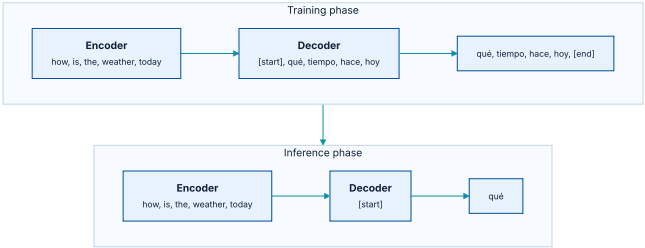


Figure 15.1 — during training the decoder sees the true target prefix; during inference it sees only what it has generated so far.

English to Spanish: 118,000 sentence pairs

PYTHON

```python
import pathlib

zip_path = keras.utils.get_file(
    origin=(
        "http://storage.googleapis.com/download.tensorflow.org/data/spa-eng.zip"
    ),
    fname="spa-eng",
    extract=True,
)
text_path = pathlib.Path(zip_path) / "spa-eng" / "spa.txt"

with open(text_path) as f:
    lines = f.read().split("\n")[:-1]

text_pairs = []
for line in lines:
    english, spanish = line.split("\t")
    spanish = "[start] " + spanish + " [end]"
    text_pairs.append((english, spanish))
```

The one non-obvious line is the wrapping of every Spanish sentence in `[start]` and `[end]`.

SECTION 15.2.1

The seed and the stop signal live in the data

OUTPUT

```
>>> import random
>>> random.choice(text_pairs)
("Who is in this room?", "[start] ¿Quién está en esta habitación? [end]")
```

`[start]` is what we feed the decoder to begin generating; `[end]` is what tells the generation loop to stop. They are **inserted in the data, not built into the model** — which is why the tokenizer must be told not to strip square brackets.

Splitting is the usual three-way shuffle: 70% train, 15% validation, 15% test, taken over the pairs rather than over the sentences, so no English sentence appears in two splits.

Two tokenizers, because punctuation is language-specific

- The characters `[` and `]` are stripped by default. We must **keep** them, so the token `"[start]"` stays distinct from the word `"start"`.
- Spanish uses `¿`, which is not in `string.punctuation`. The Spanish tokenizer needs it added to the strip set explicitly.

listing 15.9 PYTHON

```python
import string, re

strip_chars = string.punctuation + "¿"
strip_chars = strip_chars.replace("[", "").replace("]", "")

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(lowercase, f"[{re.escape(strip_chars)}]", "")
```

> For a non-toy translator you would treat punctuation as **separate tokens** rather than stripping it — otherwise the model can never produce correctly punctuated output.

The two vectorizers, and why their lengths differ

listing 15.9 (cont.) PYTHON

```python
vocab_size = 15000
sequence_length = 20

english_tokenizer = layers.TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length,
)
spanish_tokenizer = layers.TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length + 1,
    standardize=custom_standardization,
)

english_tokenizer.adapt([pair[0] for pair in train_pairs])
spanish_tokenizer.adapt([pair[1] for pair in train_pairs])
```

The Spanish sequence is **one token longer**: 20 tokens of input and 20 of label, cut from 21. And `adapt()` runs on `train_pairs` only — never on validation or test text.

Shifting the target, and masking the padding

listing 15.10 PYTHON

```python
def format_dataset(eng, spa):
    eng = english_tokenizer(eng)
    spa = spanish_tokenizer(spa)
    features = {"english": eng, "spanish": spa[:, :-1]}
    labels = spa[:, 1:]
    sample_weights = labels != 0
    return features, labels, sample_weights
```

`spa[:, :-1]` is the decoder's input; `spa[:, 1:]` is its label. The same tensor, offset by one — exactly the Shakespeare setup, with the encoder input added alongside.

> `sample_weights = labels != 0` tells Keras to **ignore padded positions** when computing loss and metrics. Without it, a model that learns only to predict padding would score well.

SECTION 15.2.1

What comes out of the pipeline

OUTPUT

```
>>> inputs, targets, sample_weights = next(iter(train_ds))
>>> print(inputs["english"].shape)
(64, 20)
>>> print(inputs["spanish"].shape)
(64, 20)
>>> print(targets.shape)
(64, 20)
>>> print(sample_weights.shape)
(64, 20)
```

Four aligned tensors of the same shape. Two go in as a dictionary of named inputs, one is the label, one is the per-position weight. Every seq2seq pipeline in this chapter has this shape.

Why the naive single-RNN approach cannot work

```python
inputs = keras.Input(shape=(sequence_length,), dtype="int32")
x = layers.Embedding(input_dim=vocab_size, output_dim=128)(inputs)
x = layers.LSTM(32, return_sequences=True)(x)
outputs = layers.Dense(vocab_size, activation="softmax")(x)
model = keras.Model(inputs, outputs)
```

Because of the step-by-step nature of RNNs, this model sees only source tokens 0…N when predicting target token N.

> *"I will bring the bag to you" becomes "Te traeré la bolsa." The **first** Spanish word corresponds to the **last** English word. There is no way to emit it without having read to the end of the source.*
>
> Section 15.2.2

So read the whole source first

Figure 15.2 — everything the decoder knows about English arrives through one 1,024-dimensional vector.

Instead of the zero initial state used in the Shakespeare generator, the decoder starts from a state that **encodes the source sentence**.

Encoder bidirectional, decoder emphatically not

listing 15.11 - encoder PYTHON

```python
embed_dim = 256
hidden_dim = 1024

source = keras.Input(shape=(None,), dtype="int32", name="english")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(source)
rnn_layer = layers.GRU(hidden_dim)
rnn_layer = layers.Bidirectional(rnn_layer, merge_mode="sum")
encoder_output = rnn_layer(x)
```

listing 15.12 - decoder PYTHON

```python
target = keras.Input(shape=(None,), dtype="int32", name="spanish")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(target)
rnn_layer = layers.GRU(hidden_dim, return_sequences=True)
x = rnn_layer(x, initial_state=encoder_output)
x = layers.Dropout(0.5)(x)
target_predictions = layers.Dense(vocab_size, activation="softmax")(x)
seq2seq_rnn = keras.Model([source, target], target_predictions)
```

Bidirectional in the encoder is a good idea — we never predict source tokens, so there is nothing to cheat at. **Bidirectional in the decoder would break training** for exactly the reason the Shakespeare experiment showed.

SECTION 15.2.2

Thirty-five million parameters, and where they went

OUTPUT

```
>>> seq2seq_rnn.summary()
| embedding_1 (Embedding)  | (None, None, 256)  |   3,840,000 |
| embedding_2 (Embedding)  | (None, None, 256)  |   3,840,000 |
| bidirectional            | (None, 1024)       |   7,876,608 |
| gru_2 (GRU)              | (None, None, 1024) |   3,938,304 |
| dropout_1 (Dropout)      | (None, None, 1024) |           0 |
| dense_1 (Dense)          | (None, None, 15000)|  15,375,000 |
 Total params: 34,869,912 (133.02 MB)
```

The output `Dense` alone is 15 million parameters — 1,024 hidden units times a 15,000-word vocabulary. **The vocabulary projection dominates**, which is a constant of language modelling at every scale.

SECTION 15.2.2

65% — and why that number should not be trusted

```python
seq2seq_rnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    weighted_metrics=["accuracy"],
)
seq2seq_rnn.fit(train_ds, epochs=15, validation_data=val_ds)
```

We reach **65% next-token accuracy**. But next-token accuracy is a poor metric for translation, and it is worth being precise about why.

- It assumes the correct target tokens 0…N are **already known** when predicting token N+1. At inference, they are not — you generate them.
- It punishes a perfectly good translation that happens to use a different word order from the reference.
- The standard alternative is **BLEU**, which compares against a set of high-quality reference translations and tolerates misalignment.

What the RNN translator actually produces

OUTPUT

```
-
I will tell you tomorrow.
[start] te lo voy mañana a decir [end]
-
I think they're happy.
[start] yo creo que son felices [end]
```

Decent for a toy, and still making basic mistakes — the first example has the adverb wedged into the middle of the verb phrase.

> This inference loop is also **inefficient**: it reprocesses the entire source and the entire generated target every time it samples one new word. A real system caches the state that has not changed.

Two limits that no amount of tuning removes

The bottleneck

The **entire** source representation must fit in the encoder's state vector, which caps the size and complexity of sentence you can translate at all.

Forgetting

RNNs progressively forget the past. By the 100th token, little information about the start of the sequence remains.

Recurrent networks dominated seq2seq in the mid-2010s — **Google Translate circa 2017 was a stack of seven large LSTM layers**, essentially the model we just built. These two limits are what drove the search for something else.

03

The Transformer architecture

Attention is derived from the problem it solves, then made into everything.

The finding is in the title

> ***Attention Is All You Need***
>
> Vaswani et al., NeurIPS 2017 — arxiv.org/abs/1706.03762

The authors were working on translation systems like the one we just built. Attention itself was not new — it had been in NLP systems for a couple of years. The **surprise** was that it was useful enough to be the *only* mechanism passing information across a sequence.

No recurrent layers at all. This finding unleashed a revolution in natural language processing, and then beyond it — vision, audio, protein structure, and the models in chapters 16 and 17.

A thought experiment about this book

Suppose you had to build a weather-prediction model using only this textbook.

- You might read the whole book cover to cover — over 100,000 words, far longer than any sequence we have handled.
- But when you actually wrote the code, you would pay **special attention** to the timeseries chapter, and within it to a few specific listings.
- You would not be worried about the details of image convolutions.

Humans are **selective and contextual** about where they pull information from. An RNN has no mechanism to refer back directly — all information must pass through the cell state, in a loop, through every intervening position.

> It is like finishing the book, closing it, and writing the weather model **entirely from memory**.

SECTION 15.3.1

Score every source vector against the current target

The goal: give the model a way to score every source vector by its relevance to the word being predicted. Assume for a moment a function `score(target_vector, source_vector)`.

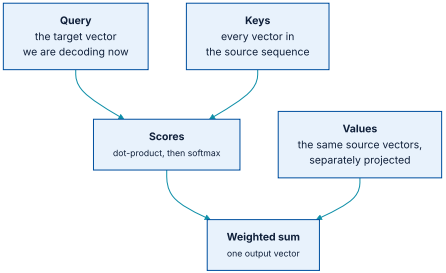


Figure 15.4 — attention assigns a relevance score to each source vector, for each target vector.

SECTION 15.3.1

The scoring function is a dot product, and the sum is a softmax

PYTHON

```python
def dot_product_attention(target, source):
    scores = np.einsum("btd,bsd->bts", target, source)
    scores = softmax(scores, axis=-1)
    return np.einsum("bts,bsd->btd", scores, source)

dot_product_attention(target, source)
```

The `einsum` subscripts spell out the shapes: `b`atch, `t`arget length, `s`ource length, `d`imension. The first contraction produces a **(batch, target, source) score matrix**; the second uses it to take a weighted sum.

The score matrix is two-dimensional

TARGET ↓ / SOURCE →	I	WILL	BRING	THE	BAG	TO	YOU
Te	·	·	·	·	·	·	**0.8**
traeré	0.2	**0.5**	**0.3**	·	·	·	·
la	·	·	·	**0.6**	0.3	·	·
bolsa	·	·	·	0.2	**0.7**	·	·
[end]	·	·	·	·	·	0.3	0.4

Read a row as: *when producing this Spanish word, how much did the model draw on each English word?* The `Te` row peaking at `you` is exactly the long-range dependency the RNN could not express.

Give the model parameters: query, key, value

PYTHON

```
query_dense = layers.Dense(dim)
key_dense = layers.Dense(dim)
value_dense = layers.Dense(dim)
output_dense = layers.Dense(dim)

def parameterized_attention(query, key, value):
    query = query_dense(query)
    key = key_dense(key)
    value = value_dense(value)
    scores = np.einsum("btd,bsd->bts", query, key)
    scores = softmax(scores, axis=-1)
    outputs = np.einsum("bts,bsd->btd", scores, value)
    return output_dense(outputs)

parameterized_attention(query=target, key=source, value=source)
```

`sum(score(target, source) * source)` has become `sum(score(query, key) * value)`. The three-argument form is more general: in rare cases you want different vectors for **scoring** and for **summing**.

The names come from search engines

Query
Your search term. In attention: the target vector for the position currently being decoded.

Keys
The photo tags used to match against the query. In attention: the projected source vectors that get scored.

Values
The photos themselves — what you actually retrieve. In attention: the projected source vectors that get summed.

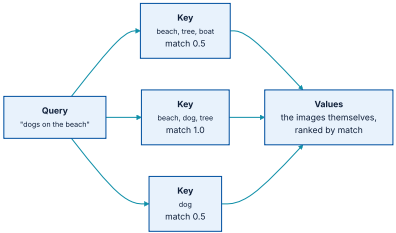


Figure 15.6 — the query is compared to keys, and the match scores rank the values.

Scale the scores before the softmax

The fix is one division:

```python
scores = softmax(scores / math.sqrt(head_dim), axis=-1)
```

Scaling by the square root of the vector length works well for any vector size — the variance of a dot product of d-dimensional vectors grows with d, so dividing by $\sqrt{d}$ holds it roughly constant. This is why the mechanism is called **scaled dot-product attention**.

One softmax sum is too blunt

The trick that works: run the whole attention operation **several times in parallel**, with different parameters. Each run is an *attention head*.

> By projecting query and key differently, one head might learn to match the **subject** of the source sentence, while another attends to **punctuation**.

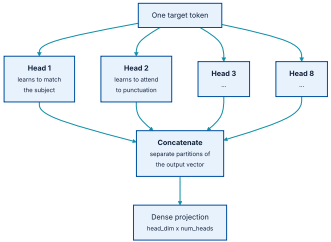


Figure 15.7 — each head attends to different parts of the source, in separate partitions of the eventual output vector.

Multi-head attention, written out

PYTHON

```python
query_dense = [layers.Dense(head_dim) for i in range(num_heads)]
key_dense = [layers.Dense(head_dim) for i in range(num_heads)]
value_dense = [layers.Dense(head_dim) for i in range(num_heads)]
output_dense = layers.Dense(head_dim * num_heads)

def multi_head_attention(query, key, value):
    head_outputs = []
    for i in range(num_heads):
        q = query_dense[i](query)
        k = key_dense[i](key)
        v = value_dense[i](value)
        scores = np.einsum("btd,bsd->bts", q, k)
        scores = softmax(scores / math.sqrt(head_dim), axis=-1)
        head_outputs.append(np.einsum("bts,bsd->btd", scores, v))
    outputs = ops.concatenate(head_outputs, axis=-1)
    return output_dense(outputs)
```

Each head produces a `head_dim`-wide output; concatenating `num_heads` of them and projecting once gives the layer's output. In practice the loop is one batched matmul.

SECTION 15.3.1

In Keras, this is one layer

```python
multi_head_attention = keras.layers.MultiHeadAttention(
    num_heads=num_heads,
    head_dim=head_dim,
)
multi_head_attention(query=target, key=source, value=source)
```

We derived it by hand first for a reason: `MultiHeadAttention` is the single most consequential layer in modern deep learning, and reading its arguments — `query`, `key`, `value`, `attention_mask`, `use_causal_mask` — is a skill the rest of this course assumes.

Now point the mechanism at its own input

```python
multi_head_attention(query=source, key=source, value=source)
```

This is **self-attention**. Each token attends to every token in its own sequence, including itself, producing a representation of the word **in context**.

> *"The train left the station on time." What kind of station? A radio station? The International Space Station? Self-attention lets the model give a high score to the pair (**station**, **train**), summing the representation of train into that of station.*
>
> Section 15.3.2

Attention alone collapses to a matrix multiplication

`MultiHeadAttention` combines *linear* projections of source elements. That is all it does. It is a very expressive **pooling** operation, and pooling is not enough.

> Consider a sequence of length one. The score matrix is a single 1, and the layer reduces to a linear projection. You could stack **100** such layers and still simplify the whole computation to **one matrix multiplication**.

Every recurrent cell eventually passes each token's vector through a dense projection with an activation. The Transformer adds this back in the simplest possible way: a feedforward network of **two Dense layers with a nonlinearity between them**.

SECTION 15.3.2

The division of labour inside a Transformer block

Attention
Passes information **across** the sequence. Mixes positions. Contains no nonlinearity.

Feedforward
Updates the representation of **each item independently**. Mixes features, not positions. Contains the nonlinearity.

Two more ingredients come straight from the ConvNet chapters: **residual connections** and **normalization**, both of which chapter 9 established as essential to training anything deep.

The encoder block: attend, add, norm; feed forward, add, norm.

The encoder block: what it holds

listing 15.14 - constructor PYTHON

```python
class TransformerEncoder(keras.Layer):
    def __init__(self, hidden_dim, intermediate_dim, num_heads):
        super().__init__()
        key_dim = hidden_dim // num_heads
        self.self_attention = layers.MultiHeadAttention(num_heads, key_dim)
        self.self_attention_layernorm = layers.LayerNormalization()
        self.feed_forward_1 = layers.Dense(intermediate_dim, activation="relu")
        self.feed_forward_2 = layers.Dense(hidden_dim)
        self.feed_forward_layernorm = layers.LayerNormalization()
```

`key_dim = hidden_dim // num_heads` keeps the total width constant as heads are added — eight heads of 32 dimensions rather than one head of 256. **Splitting, not growing.**

The encoder block: what it does

listing 15.14 - call PYTHON

```python
    def call(self, source, source_mask):
        residual = x = source
        mask = source_mask[:, None, :]
        x = self.self_attention(query=x, key=x, value=x, attention_mask=mask)
        x = x + residual
        x = self.self_attention_layernorm(x)

        residual = x
        x = self.feed_forward_1(x)
        x = self.feed_forward_2(x)
        x = x + residual
        x = self.feed_forward_layernorm(x)
        return x
```

Input and output have the **same shape**, so blocks stack — each one building a progressively more expressive representation of the source sentence. RoBERTa, later in this chapter, stacks twelve.

LayerNormalization, not BatchNormalization

layer normalization — PYTHON

```python
# input shape: (batch_size, sequence_length, embedding_dim)
def layer_normalization(batch_of_sequences):
    mean = np.mean(batch_of_sequences, keepdims=True, axis=-1)
    variance = np.var(batch_of_sequences, keepdims=True, axis=-1)
    return (batch_of_sequences - mean) / variance
```

batch normalization — PYTHON

```python
# input shape: (batch_size, height, width, channels)
def batch_normalization(batch_of_images):
    mean = np.mean(batch_of_images, keepdims=True, axis=(0, 1, 2))
    variance = np.var(batch_of_images, keepdims=True, axis=(0, 1, 2))
    return (batch_of_images - mean) / variance
```

`BatchNormalization` pools over **axis 0**, creating interactions between samples in a batch. `LayerNormalization` pools only over the last axis, normalising each sequence **independently** — which is what variable-length sequence data requires.

The padding mask, and its shape

We use it to stop any token from attending to **padding**, which carries no information.

PYTHON

```python
# source_mask marks the non-padding tokens: (batch_size, source_length)
source_mask = source != 0

# upranked inside the layer to broadcast across every target position
mask = source_mask[:, None, :]   # (batch_size, 1, source_length)
```

The `None` in the middle is the whole trick: one mask, broadcast across every row of the score matrix, because *which source tokens are padding* does not depend on which target position is asking.

SECTION 15.3.3

The decoder attends twice

1. **Self-attention** over the target sequence, so each target position can use information from other target positions — masked so it can only look backward.
2. **Cross-attention**, whose query is the target and whose key and value are the encoder output — this is what brings information across from the source language.
3. **Feedforward**, same as the encoder.

The decoder block: masked self-attention, then cross-attention, then feedforward — each with add-and-norm.

The decoder block: what it holds

listing 15.15 - constructor PYTHON

```python
class TransformerDecoder(keras.Layer):
    def __init__(self, hidden_dim, intermediate_dim, num_heads):
        super().__init__()
        key_dim = hidden_dim // num_heads
        self.self_attention = layers.MultiHeadAttention(num_heads, key_dim)
        self.self_attention_layernorm = layers.LayerNormalization()
        self.cross_attention = layers.MultiHeadAttention(num_heads, key_dim)
        self.cross_attention_layernorm = layers.LayerNormalization()
        self.feed_forward_1 = layers.Dense(intermediate_dim, activation="relu")
        self.feed_forward_2 = layers.Dense(hidden_dim)
        self.feed_forward_layernorm = layers.LayerNormalization()
```

Nine attributes, three of them `LayerNormalization` — one per residual junction. Counting normalization layers is a quick way to check a Transformer implementation by eye.

The decoder block: what it does

listing 15.15 - call PYTHON

```
    def call(self, target, source, source_mask):
        residual = x = target
        x = self.self_attention(query=x, key=x, value=x, use_causal_mask=True)
        x = self.self_attention_layernorm(x + residual)

        residual = x
        mask = source_mask[:, None, :]
        x = self.cross_attention(
            query=x, key=source, value=source, attention_mask=mask
        )
        x = self.cross_attention_layernorm(x + residual)

        residual = x
        x = self.feed_forward_1(x)
        x = self.feed_forward_2(x)
        x = self.feed_forward_layernorm(x + residual)
        return x
```

`use_causal_mask=True` on self-attention stops the decoder seeing its own future. The padding `attention_mask` on cross-attention stops it attending to empty source positions. **Different problems, different masks.**

Attention is bidirectional by default — which would be fatal

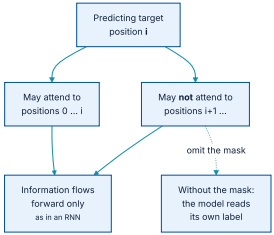
The fix is a lower-triangular attention mask:

OUTPUT

```
[
    [1, 0, 0, 0, 0],
    [1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0],
    [1, 1, 1, 1, 0],
    [1, 1, 1, 1, 1],
]
```

Row *i* is the mask for target position *i*. Row 0: the first token may attend only to itself. Row 1: the second may attend to the first and itself. And so on — the same forward-only information flow an RNN gets for free from its structure.

One argument, and a bug you will not be warned about



The same forward-only information flow an RNN gets free from its structure, imposed here by hand.

In Keras this is one argument: `use_causal_mask=True`. **Forgetting it is one of the most common and most silent bugs in hand-written Transformer code.**

Assembling the translator: the encoder half

listing 15.16 - encoder PYTHON

```python
hidden_dim = 256
intermediate_dim = 2048
num_heads = 8

source = keras.Input(shape=(None,), dtype="int32", name="english")
x = layers.Embedding(vocab_size, hidden_dim)(source)
encoder_output = TransformerEncoder(hidden_dim, intermediate_dim, num_heads)(
    source=x,
    source_mask=source != 0,
)
```

`source != 0` computes the padding mask inline — no separate layer, just a Boolean tensor built from the input.

Assembling the translator: the decoder half

listing 15.16 - decoder PYTHON

```
target = keras.Input(shape=(None,), dtype="int32", name="spanish")
x = layers.Embedding(vocab_size, hidden_dim)(target)
x = TransformerDecoder(hidden_dim, intermediate_dim, num_heads)(
    target=x,
    source=encoder_output,
    source_mask=source != 0,
)
x = layers.Dropout(0.5)(x)
target_predictions = layers.Dense(vocab_size, activation="softmax")(x)
transformer = keras.Model([source, target], target_predictions)
```

One encoder block and one decoder block — the 2017 paper used **six of each**. Everything else, down to the final softmax over the Spanish vocabulary, is unchanged from the GRU model.

SECTION 15.3.4

Fourteen million parameters — less than half the RNN

OUTPUT

```
>>> transformer.summary()
| embedding_5 (Embedding)  | (None, None, 256)   |   3,840,000 |
| transformer_encoder_1    | (None, None, 256)   |   1,315,072 |
| embedding_6 (Embedding)  | (None, None, 256)   |   3,840,000 |
| transformer_decoder_1    | (None, None, 256)   |   1,578,752 |
| dropout_9 (Dropout)      | (None, None, 256)   |           0 |
| dense_11 (Dense)         | (None, None, 15000) |   3,855,000 |
 Total params: 14,428,824 (55.04 MB)
```

34.9 M for the RNN against **14.4 M** here. The encoder and decoder blocks together are under 3 M parameters — the embeddings and the vocabulary projection dominate, as always.

58%. Worse than the RNN. Can you spot why?

PYTHON

```
transformer.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    weighted_metrics=["accuracy"],
)
transformer.fit(train_ds, epochs=15, validation_data=val_ds)
```

65%
GRU seq2seq

58%
Transformer, first attempt

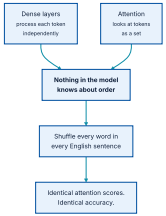
Seven percentage points **worse**. Either the architecture is not what it was hyped up to be, or something is missing from the implementation.

> Stop here. Give the room two minutes with the code on the previous slide before turning over.

What we built is not a sequence model at all

It is composed of dense layers that process tokens independently, and an attention layer that looks at tokens **as a set**.

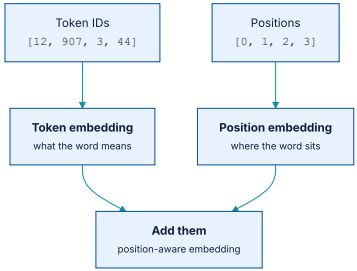
Change the order of the tokens and you get identical pairwise attention scores and identical context-aware representations. Shuffle every word of every English source sentence and the model **would not notice**. You would get the same accuracy.



Attention is a set-processing mechanism: it sees relationships between pairs, and is blind to where in the sequence they sit.

For RNNs, order-awareness came free from the layer's computation. For the Transformer, we must **inject positional information into the embeddings themselves**.

Positional embedding: add where to what



Two embedding tables, added together. The model works out how to use the extra information.

The most obvious scheme — concatenate the raw integer position — is a poor one: positions can be large integers, and neural networks dislike large input values and discrete input distributions.

Sinusoidal, or learned

Sinusoidal (the 2017 paper)

Add a vector of values in `[-1, 1]` that varies **cyclically** with position, built from cosine functions. Clever: it characterises any integer in a large range using only small values.

Learned (what we use)

Learn positional vectors the same way we learn word embeddings — an `Embedding` table indexed by position. **Simpler and more effective** in practice.

The learned scheme has one cost: it fixes a **maximum sequence length** at build time, since the table has one row per position. Sinusoidal encodings extrapolate; learned ones do not.

The PositionalEmbedding layer

listing 15.17 PYTHON

```python
from keras import ops

class PositionalEmbedding(keras.Layer):
    def __init__(self, sequence_length, input_dim, output_dim):
        super().__init__()
        self.token_embeddings = layers.Embedding(input_dim, output_dim)
        self.position_embeddings = layers.Embedding(sequence_length, output_dim)

    def call(self, inputs):
        positions = ops.cumsum(ops.ones_like(inputs), axis=-1) - 1
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions
```

`ops.cumsum(ops.ones_like(inputs), axis=-1) - 1` produces `[0, 1, 2, ...]` for every sequence in the batch — a backend-agnostic way to write `arange` that works under TensorFlow, PyTorch, and JAX alike.

The layer is a **drop-in replacement** for `Embedding`.

Two lines changed

listing 15.18 PYTHON

```python
source = keras.Input(shape=(None,), dtype="int32", name="english")
x = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)(source)
encoder_output = TransformerEncoder(hidden_dim, intermediate_dim, num_heads)(
    source=x,
    source_mask=source != 0,
)

target = keras.Input(shape=(None,), dtype="int32", name="spanish")
x = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)(target)
x = TransformerDecoder(hidden_dim, intermediate_dim, num_heads)(
    target=x,
    source=encoder_output,
    source_mask=source != 0,
)
x = layers.Dropout(0.5)(x)
target_predictions = layers.Dense(vocab_size, activation="softmax")(x)
transformer = keras.Model([source, target], target_predictions)
```

Everything else is untouched. `layers.Embedding` became `PositionalEmbedding`, twice.

67% — and a third of the epoch time

65%	58%	67%
GRU seq2seq, 34.9 M params	Transformer without positions	Transformer with positions, 14.4 M params

A noticeable improvement on the GRU — and all the more so given that this model has **half the parameters**.

There is a second thing to notice about the training run: each epoch takes about **one third** the time. With attention there is no looped state passing, so on a GPU or TPU the entire attention computation happens **in one go**.

> That speed-up is not a minor convenience. It is the property that made scaling to billions of parameters economically possible — the subject of chapter 16.

The Transformer's translations

OUTPUT

```
-
I'll see you at the library tomorrow.
[start] te veré en la biblioteca mañana [end]
-
Do you know how to ride a bicycle?
[start] sabes montar en bici [end]
-
Tom didn't want to do their dirty work.
[start] tom no quería hacer su trabajo [end]
-
Is he back already?
[start] ya ha vuelto [end]
```

Subjectively much better than the GRU translations. It is still a toy model — but it is a **better toy model**.

None of these choices is provably optimal

The answer is simple — it is not.

- Many improvements to attention, normalization, and positional embeddings have been proposed since 2017.
- Much current research replaces attention with something **less computationally complex**, as sequence lengths grow very long.
- Something will eventually supplant the Transformer — possibly before you finish this course.

> The field moves **empirically**. Attention grew out of an attempt to augment RNNs; after years of guessing and checking by a great many people, it gave rise to the Transformer. There is little reason to think that process is done.

04

Classification with a pretrained Transformer

124 million parameters, one epoch of fine-tuning, and a three-point jump.

SECTION 15.4

Two properties that invited scaling

Faster to train

No loops during training, which is always good on a GPU or TPU. The whole sequence is processed at once.

Data hungry

We saw this ourselves: the RNN plateaued on validation after about 5 epochs; the Transformer was **still improving after 30**.

These observations prompted many to try scaling the Transformer up — more data, more layers, more parameters — with strikingly good results.

The result was a distinctive shift in the field toward **large pretrained models** that can cost millions to train but perform noticeably better across a wide range of text problems.

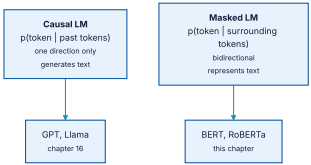
Masked language modelling

But BERT was 100 to 300 million parameters, against our 14 million. That needs a great deal of training data, so the authors used a variant of the language-model objective.

1. Take a sequence of text and replace about **15% of tokens** with a special `[MASK]` token.
2. Train the model to predict the **original** value of each masked token.
3. Note what is *not* needed: **any labels at all**. For any text sequence you can choose random tokens and mask them.

> A causal language model learns p(token | **past** tokens). A masked language model learns p(token | **surrounding** tokens).

Encoder or decoder — a choice that follows from the objective



The objective determines the masking, the masking determines what the model is good for.

	CAUSAL LM	MASKED LM
Distribution	p(token \{}| past)	p(token \{}| surrounding)
Attention	Causal-masked, one direction	Bidirectional
Good at	**Generating** text in a loop	**Representing** text richly
Examples	GPT, Llama — chapter 16	BERT, RoBERTa — this section

Pretrained word embeddings were already common practice when BERT appeared — chapter 14 did exactly that. Pretraining a whole Transformer gave something more powerful: an embedding for a word **in the context of the words around it**.

Same architecture, ten times the data

16 GB
BERT — mainly Wikipedia

160 GB
RoBERTa — text from across the web

~$300k
estimated training cost at the time

Because of the extra data, RoBERTa performs noticeably better **at an equivalent parameter count**. That relationship — data quantity buying quality at fixed model size — is the central empirical fact of the pretraining era.

Three things you need to use a pretrained model

A matching tokenizer

Text must be tokenized **exactly** as it was during pretraining. If words map to different indices, the learned representations are meaningless.

A matching architecture

The internal maths must be recreated exactly. Ours nearly matches `TransformerEncoder` already — but *nearly* is not enough.

The pretrained weights

Produced by training for about a day on **1,024 GPUs** over billions of words.

Recreating the first two by hand would be possible but time-consuming and error-prone. As in chapter 8, we use **KerasHub** instead.

Two lines to load a pretrained Transformer

listing 15.20 PYTHON

```python
import keras_hub

tokenizer = keras_hub.models.Tokenizer.from_preset("roberta_base_en")
backbone = keras_hub.models.Backbone.from_preset("roberta_base_en")
```

`from_preset()` loads weights, configuration, and tokenizer assets together — which is precisely what keeps the three requirements on the previous slide consistent with one another.

OUTPUT

```
>>> tokenizer("The quick brown fox")
Array([ 133, 2119, 6219, 23602], dtype=int32)
```

RoBERTa's tokenizer is close to the subword tokenizer built in chapter 14, with tweaks to handle Unicode from any language.

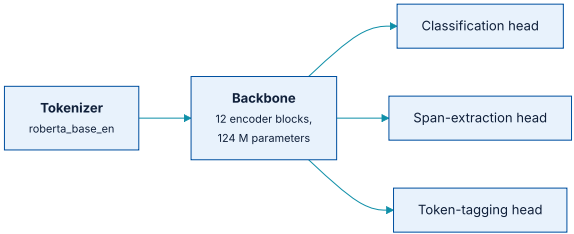
The tokenization decision, at 160 GB

CHOICE	WHAT GOES WRONG AT THIS SCALE
Character level	Input sequences become far too long, making the model much more expensive to train — attention cost grows with the *square* of length.
Word level	Covering the distinct words across millions of web documents blows up the vocabulary, making the front `Embedding` layer unworkably large.
Subword (BPE)	Handles **any** word with a 50,000-term vocabulary. This is why every large model uses it.

The trade-off from chapter 14 has not changed — but the pressure on both ends is far greater, which is what makes the middle option not merely convenient but **required**

A model without a head

What we loaded maps an input sequence to an output sequence of shape **(batch_size, sequence_length, 768)**. It is not set up for any particular loss function.



One backbone, many heads: classifying sentences, extracting spans, tagging parts of speech.

> Think of it as attaching different heads to a screwdriver — a Phillips head for one task, a flat head for another.

SECTION 15.4.2

124 million parameters, twelve blocks deep

OUTPUT

```
>>> backbone.summary()
Model: "roberta_backbone"
| token_ids (InputLayer)      | (None, None)      |          0 |
| embeddings                  | (None, None, 768) | 38,996,736 |
|  (TokenAndPositionEmbedding)|                   |            |
| embeddings_layer_norm       | (None, None, 768) |      1,536 |
| padding_mask (InputLayer)   | (None, None)      |          0 |
| transformer_layer_0         | (None, None, 768) |  7,087,872 |
| transformer_layer_1         | (None, None, 768) |  7,087,872 |
| ...                         | ...               |        ... |
| transformer_layer_11        | (None, None, 768) |  7,087,872 |
 Total params: 124,052,736 (473.22 MB)
```

Twelve identical encoder blocks stacked — exactly the stacking property we noted when the `TransformerEncoder` preserved its input shape. This is the **smallest** RoBERTa checkpoint, and the largest model used anywhere in this book so far.

Packing tokens the way pretraining did

OUTPUT

```
[
    ["<s>", "the", "quick", "brown", "fox", "jumped", ".", "</s>"],
    ["<s>", "the", "panda", "slept", ".", "</s>", "<pad>", "<pad>"],
]
```

listing 15.21 PYTHON

```python
def preprocess(text, label):
    packer = keras_hub.layers.StartEndPacker(
        sequence_length=512,
        start_value=tokenizer.start_token_id,
        end_value=tokenizer.end_token_id,
        pad_value=tokenizer.pad_token_id,
        return_padding_mask=True,
    )
    token_ids, padding_mask = packer(tokenizer(text))
    return {"token_ids": token_ids, "padding_mask": padding_mask}, label

preprocessed_train_ds = train_ds.map(preprocess)
```

Matching the pretraining token ordering as closely as possible makes the model train **faster and more accurately**. The IMDb loading code from chapter 14 is reused unchanged.

SECTION 15.4.3

One preprocessed batch

OUTPUT

```
>>> next(iter(preprocessed_train_ds))
({"token_ids": <tf.Tensor: shape=(16, 512), dtype=int32, numpy=
  array([[   0,  713,   56, ...,    1,   1,  1],
         [   0, 1121,    5, ...,  101,  24,  2],
         ...,
         [   0,  734,    8, ...,    1,   1,  1]], dtype=int32)>,
  "padding_mask": <tf.Tensor: shape=(16, 512), dtype=bool, numpy=
  array([[ True,  True,  True, ..., False, False, False],
         [ True,  True,  True, ...,  True,  True,  True],
         ...,
         [ True,  True,  True, ..., False, False, False]])>},
 <tf.Tensor: shape=(16,), dtype=int32, numpy=array([0, 1, ...])>)
```

Batch size is now **16**, not the 32 or 64 used earlier — a 124-million-parameter model with 512-token sequences is a different memory proposition. Rows ending in `1` are padded; the `padding_mask` marks exactly where.

Where does pretraining data come from?

MODEL	PRETRAINING DATA
The first Transformer (2017)	A well-known English-German translation set, **4 million sentence pairs**.
BERT (2018)	A dump of English Wikipedia plus a dataset of **7,000 self-published books**.
GPT-2 (2019)	Scraped by following **outgoing links from Reddit**.
Llama (latest)	"**15 trillion tokens** of data from publicly available sources."

The short answer is: the internet. The other answer is that this has increasingly become a **secret** — companies often do not release the data or the precise mixture of sources.

> When possible, pay close attention to where a model's data came from. It shapes both the **biases** and the **performance** of everything built on top of it.

The classification head, and why it uses token zero

listing 15.22 PYTHON

```python
inputs = backbone.input
x = backbone(inputs)
x = x[:, 0, :]
x = layers.Dropout(0.1)(x)
x = layers.Dense(768, activation="relu")(x)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
classifier = keras.Model(inputs, outputs)
```

`x[:, 0, :]` takes the **first token's** representation. Mean or max pooling would also work, but this works slightly better — and the reason is the nature of attention.

> The first position in the final encoder layer can attend to **every** other position and pull information from them. So rather than pooling with something coarse like an average, attention pools **contextually**.

One epoch, and a very small learning rate

listing 15.23 PYTHON

```python
classifier.compile(
    optimizer=keras.optimizers.Adam(5e-5),
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
classifier.fit(
    preprocessed_train_ds,
    validation_data=preprocessed_val_ds,
)
```

`5e-5` is roughly **twenty times smaller** than Adam's default. This is the fine-tuning discipline from chapter 8, unchanged: large updates would destroy the representations that cost $300,000 to learn.

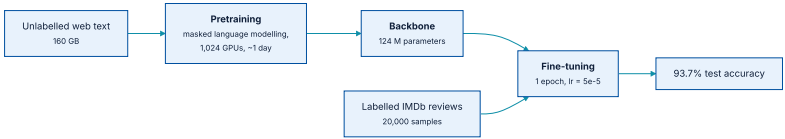
OUTPUT

```
>>> classifier.evaluate(preprocessed_test_ds)
[0.168127179145813, 0.9366399645805359]
```

93.7% test accuracy after a single epoch, against the 90% ceiling reached in chapter 14.

What the last two chapters bought, in one table

MODEL	IMDB TEST ACCURACY	COST
Bigram bag-of-words (chapter 14)	≈ 89%	seconds, on a CPU
Sequence model with pretrained embeddings	≈ 90%	minutes, one GPU
RoBERTa base, fine-tuned 1 epoch	**93.7%**	124 M params, one GPU-hour
RoBERTa large, fine-tuned	**> 95%**	300 M params



Where the accuracy actually came from: months of unlabelled pretraining, then one epoch on 20,000 labelled reviews.

This is a far more expensive model to run than the bigram classifier, and the clear benefit has to be weighed against that. **Chapter 18 makes that trade-off explicit**; for now, note that the three points cost roughly four orders of magnitude in compute.

05

What makes the Transformer effective?

A geometric answer, by way of a model from 2013.

Word2Vec, and the magic vectors

The resulting space did more than capture similarity. It showed a kind of emergent *word arithmetic*:

> $V(\text{king}) - V(\text{man}) + V(\text{woman}) \approx V(\text{queen})$
>
> A gender vector nobody trained for

There were dozens of such vectors — a plural vector, a vector from wild animals to their closest pet equivalent. **The model had not been trained for any of this explicitly.**

SECTION 15.5

How much a 2013 toy and a 2024 model have in common

	WORD2VEC	TRANSFORMER
Goal	Embed tokens in a vector space	Embed tokens in a vector space
Learning principle	Tokens that co-occur end up close	Tokens that co-occur end up close
Distance function	Cosine distance	Cosine distance
Dimensionality	~1,000 per word	1,000 to 10,000 per word

You might object: a Transformer is trained to predict missing words, not to group tokens in a space. How does that objective relate to maximising dot-products between co-occurring tokens? **The answer is the attention mechanism.**

Attention is embedding-space refinement

Attention learns a new embedding space by **linearly recombining** embeddings from a prior space, weighted toward tokens that are already close.

Because the weighting is a dot product, tokens with high similarity contribute most to each other's new representation. Over training, this **pulls already-close vectors together** — turning correlation relationships into proximity relationships.

A Transformer learns a series of incrementally refined embedding spaces, each recombining elements of the previous one.

Continuous, and interpolative

Semantically continuous

Moving a little in the embedding space changes the human-facing meaning only a little. Word2Vec's space had this property too.

Semantically interpolative

The midpoint between two points represents the **intermediate meaning** — a direct consequence of each space being built by interpolating vectors from the previous one.

This is not entirely unlike the brain. The key learning principle there is **Hebbian learning** — *neurons that fire together, wire together*. Correlations between firing events become proximity in the network.

> The Transformer, Word2Vec, and the brain all turn correlation relationships into vector-proximity relationships. All three are **maps of a space of information**.

Vector programs, not vector functions

With that representation power, and a more refined autoregressive objective, we are no longer confined to **linear** transformations like a gender vector.

WORD2VEC COULD STORE	A LARGE TRANSFORMER CAN STORE
`plural(cat) -> cats`	`write_this_in_style_of_shakespeare("...poem...")`
`male_to_female(king) -> queen`	and **millions** of such programs

These are so complex that it is more accurate to call them **vector programs** than vector functions — highly nonlinear maps of the latent space onto itself. Not Python programs: no symbolic statements, no step-by-step data processing.

A database you can retrieve more from than you put in

It is continuous

Not discrete entries but a **curve**. You can move along it to nearby points and interpolate between them — so you can retrieve much more than was put in.

It stores programs

RoBERTa holds facts, places, people, dates, relationships — but **primarily** it is a database of vector programs.

Interpolation is the same mechanism behind both outcomes: it leads to **generalization**, and it leads to **hallucination**. Not all of what you retrieve will be accurate or meaningful.

Interpolation is not synthesis

> *These models are **fundamentally interpolative**. Thanks to attention, they learn an interpolative embedding space for a significant chunk of all text written in English. Wandering that space leads to interesting, unexpected generalizations — but it cannot synthesize something fundamentally new with anything close to genuine, human-level intelligence.*
>
> Section 15.5, closing

Keep this claim in mind through chapters 16 and 19, where it is tested against much larger models and much stronger claims.

COMMON FAILURE MODES

Four ways Transformer code goes quietly wrong

No causal mask on the decoder

Training accuracy shoots up, generation produces nothing coherent. The model is reading its own label. `use_causal_mask=True`.

No positional embedding

The model trains, converges, and scores several points too low with no error message. It is treating your sentences as **bags of words**.

Tokenizer mismatched to weights

Fine-tuning a pretrained backbone with the wrong tokenizer produces near-random performance. Load both from the **same preset**.

Default learning rate on fine-tuning

Adam's default is ~20× too large for a pretrained backbone and will wash out the representations in the first few batches.

Three of these four produce **no exception and no warning** — only a number that is worse than it should be. That is what makes them worth memorising.

SUMMARY

What has to survive this chapter

1. **A language model learns p(token | past tokens)** — and generates text of any length by being called in a loop, output becoming input.
2. **A masked language model** learns p(token | surrounding tokens), which is better for representing text than for generating it.
3. **Seq2seq** splits into an encoder that represents the source and a decoder that predicts the target one token at a time.
4. **Attention** scores by dot product, normalises by softmax, and returns a weighted sum — giving direct access to any position in a sequence.

SUMMARY

What has to survive this chapter (2 of 2)

1. **The Transformer block** is attention (mixes positions, no nonlinearity) plus feedforward (mixes features, has the nonlinearity), with residuals and layer normalization around both.
2. **Attention is order-blind**, so positional embeddings are not optional — leaving them out costs several points and raises no error.
3. **Pretraining then fine-tuning** beat training from scratch decisively: 93.7% in one epoch against a 90% ceiling.
4. **Attention turns correlation into proximity**, building interpolative embedding spaces — the source of both generalization and hallucination.

NOTEBOOK

03_attention_from_scratch.ipynb

PAPER

Vaswani et al., Attention Is All You Need

NEXT

Chapter 16 — Text generation